

CIF request / logic flow — AXI to MCC

Channel Interface. Module-level sub-blocks by name; only the load-bearing seam signals shown. AXI4 front end -> validate/segment/hash -> ROB -> CDC -> MCC. Handshake shorthand: v/r=valid/ready, f/e=full/empty, oh=one-hot, bc=broadcast, ae=async CDC.

Spine (read): AR_CH -> AR_REQ_FIFO -> PORT_ARB_R -> REQ_VAL_R -> BURST_SPLIT_R -> R_ADR_HASH -> PKT_FORM_R -> ROB_RD -> CH_R_REQ_AFIFO =ae=> MCC

Spine (write): AW_CH/W_CH -> WR_PAIR -> PORT_ARB_W -> REQ_VAL_W -> BURST_SPLIT_W -> W_ADR_HASH -> PKT_FORM_W -> ROB_WR -> CH_W_REQ_AFIFO =ae=> MCC

1 Host & AXI ports (x4 ports)

module	role	interface to next (handshake + meta/data)
AXI_INTERCONNECT	SoC crossbar	OUT: routes masters onto 4 CIF slave ports; AXI5-channel v/r each
AX4IPO..3	AXI4 slave port (x4)	OUT: terminates AR/AW/W (in), R/B (out) per port
AR_CH	read-addr channel	OUT: araddr, arlen, arsize, arid arvalid/arready -> AR_REQ_FIFO
AW_CH	write-addr channel	OUT: awaddr, awlen, awsize, avid awvalid/awready -> AW_REQ_FIFO
W_CH	write-data channel	OUT: beats wdata, wstrb, wlast wvalid/wready -> WD_SRAM (via WR_SRAM_MGR); descriptor start_ptr, count -> W_BURST_FIFO
R_CH	read-data channel (out)	IN: rdata, rresp, rlast, rid rvalid/rready <- MERGE_LOGIC
BRESP_CH	write-resp channel (out)	IN: bresp, bid bvalid/bready <- RESP_DISP

2 Read request path (AR -> MCC)

module	role	interface to next (handshake + meta/data)
AR_REQ_FIFO	AR buffer (decouple ARREADY)	OUT: AR req; f/e -> ARREADY back-pressure -> PORT_ARB_R
PORT_ARB_R	1-of-4 read port arb	OUT: one AR stream (rr) -> REQ_VAL_R
REQ_VAL_R	align/len/legality, compute n_segs	OUT: valid req + n_segs -> BURST_SPLIT_R; illegal -> ERR_HDLR_R
ERR_HDLR_R	read error path	OUT: DECERR/SLVERR -> alloc ROB_RD entry, retire-eligible
BURST_SPLIT_R	cut burst into row_size segments	OUT: per-segment req (1 pkt each) -> R_ADR_HASH
R_ADR_HASH	XOR-hash row+BG+bank (col un-hashed)	IN: HASH_CFG (bc). OUT: daddr coords -> PKT_FORM_R
ROB_RD	read reorder buf start_ptr, exp_pkts, bitmap, status	OUT: rob_index + dbuf base -> PKT_FORM_R; bitmap full -> MERGE_LOGIC
RD_SRAM_MGR	alloc dbuf range at ROB alloc	OUT: dbuf_addr -> PKT_FORM_R; free on merge/drain
PKT_FORM_R	build read packet	OUT: rob_index, dbuf_addr, daddr, beats -> CH_R_PKT_FIFO
CH_R_PKT_FIFO	per-channel pkt stage	OUT: read pkt; f/e -> CH_R_REQ_AFIFO
CH_R_REQ_AFIFO	CDC CIF->MCC	OUT: read req pkt v/r (ae) -> MCC

3 Write request path (AW + W -> MCC)

module	role	interface to next (handshake + meta/data)
AW_REQ_FIFO	AW buffer	OUT: AW req f/e -> WR_PAIR
WR_SRAM_MGR	alloc per-burst slot-vector on W arrival	OUT: wd_slot[] -> WD_SRAM write + W_BURST_FIFO; free per pkt on completion
W_BURST_FIFO	write-data descriptor start_ptr, count (not beats)	OUT: descriptor -> WR_PAIR
WR_PAIR	pair AW addr with W descriptor	OUT: paired write req addr, wd_slot, count -> PORT_ARB_W
PORT_ARB_W	1-of-4 write port arb	OUT: one write stream (rr) -> REQ_VAL_W
REQ_VAL_W	legality + segmentation (n_segs)	OUT: valid req -> BURST_SPLIT_W; illegal -> ERR_HDLR_W
ERR_HDLR_W	write error path	OUT: DECERR/SLVERR -> alloc ROB_WR entry, retire-eligible
BURST_SPLIT_W	split burst, map seg -> WD_SRAM slot	OUT: per-segment req -> W_ADR_HASH
W_ADR_HASH	XOR-hash row+BG+bank	IN: HASH_CFG (bc). OUT: daddr -> PKT_FORM_W
ROB_WR	write reorder buf; SLOT_FIFO + RETIRE_LOGIC (restore AXID order)	OUT: rob_index -> PKT_FORM_W; retire -> BRESP_GEN
PKT_FORM_W	build write packet	OUT: rob_index, wd_slot, daddr, beats -> CH_W_PKT_FIFO
CH_W_PKT_FIFO	per-channel pkt stage	OUT: write pkt f/e -> CH_W_REQ_AFIFO
CH_W_REQ_AFIFO	CDC CIF->MCC	OUT: write req pkt v/r (ae) -> MCC

4 Data SRAM & managers (dual-domain: CIF + MCC ports)

module	role	interface to next (handshake + meta/data)
RD_SRAM	512b/line read buffer	IN: MCC writes returned read data @ dbuf_addr (MC port). OUT: CIF reads in segment order -> MERGE_LOGIC (CIF port)
WD_SRAM	512b/line write buffer	IN: CIF writes W beats @ wd_slot (CIF port). OUT: MCC reads @ wd_slot to drive DFI (MC port)

5 Response path (MCC -> master)

module	role	interface to next (handshake + meta/data)
CH_R_RESP_AFIFO	CDC MCC->CIF read completion	OUT: rob_index, pkt_num, status v/r (ae) -> ROB_RD
ROB_RD (return)	set bitmap bit per pkt	OUT: bitmap full -> trigger MERGE_LOGIC
MERGE_LOGIC	wait all pkts, drain RD_SRAM in segment order	OUT: rdata, rresp, rlast v/r -> R_CH; free dbuf range (RD_SRAM_MGR)
CH_W_RESP_AFIFO	CDC MCC->CIF write completion	OUT: rob_index, pkt_num, status v/r (ae) -> ROB_WR
ROB_WR (retire)	mark done, restore AXID order	OUT: retired write -> BRESP_GEN; free wd_slot (WR_SRAM_MGR)
BRESP_GEN	form AXI write response	OUT: bresp, bid -> RESP_DISP
RESP_DISP	1-of-4 port demux	OUT: bresp, bid bvalid/bready -> BRESP_CH[port]

6 Config (APB / CSR — broadcast, off the request path)

module	role	interface to next (handshake + meta/data)
APB_SLAVE	host register access	OUT: reg r/w -> CSR
CSR	control/status word file	OUT: CFG_REG, ADR_CFG, GEOM_CFG, HASH_CFG (bc)
CFG_REG / GEOM_CFG	enable/mode + ranks/BG/banks/rows/cols, row_size	OUT (bc): -> REQ_VAL, BURST_SPLIT (n_segs, row_size)
ADR_CFG / HASH_CFG	field widths/positions + XOR taps	OUT (bc): -> R/W_ADR_HASH, address decode

Seams to MCC: req out = CH_R/W_REQ_AFIFO ({rob_index, daddr, dbuf_addr|wd_slot, beats}); resp in = CH_R/W_RESP_AFIFO ({rob_index, pkt_num, status}); data shared via RD_SRAM (MCC writes) + WD_SRAM (MCC reads). Column left un-hashed so MCC address-slice stays pure. Mirrors `mc_request_flow.pdf` on the MCC side.